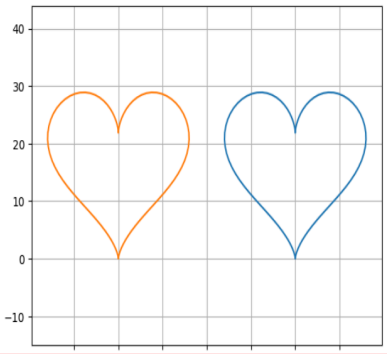
```python
# การสะท้อนธรรมดา
A1 = np.array([[-1,0],[0,1]])
theta2 = np.linspace(0,2 * np.pi, 100)
X_v2 = 16 * (np.sin(theta2) ** 3) + 20
Y_v2 = 13 * np.cos(theta2) - 5 * np.cos(2 * theta2) - 2 * np.cos(3 * theta2) - np.cos(4 * theta2) + 17
X2 = np.row_stack((X_v2, Y_v2))
ax2 = tranform(A1, X2)
plt.axis("equal")
plt.grid(True)
plt.plot(X2[0,:], X2[1,:])
plt.plot(ax2[0,:], ax2[1,:])
plt.show()
```

```
C:\Users\piyaw\AppData\Local\Temp\ipykernel_12916\1686745983.py:6: DeprecationWarning: `row_stack` alia
  X2 = np.row_stack((X_v2, Y_v2))
```




```python
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

def tranform(A, X):
    return np.dot(A,X)

A = np.array([[1,1],[0,-1]])
X = np.column_stack([[2,3],[0,1],[0,5],[-1,6]])
ax = tranform(A, X)
print(ax)
```

```python
import numpy as np

def transform(A, X):
    """คำนวณการแปลงเชิงเส้น AX (A ขนาด 2x2, X ขนาด 2xn)"""
    return np.dot(A, X)

A = np.array([[1, 1],
              [0, -1]])

# จุด X (เขียนแบบคอลัมน์เวกเตอร์)
X = np.column_stack([[2, 3]])
ax = transform(A, X)

print(A, "maps\n", X, "to\n", ax)
```

คำนวณการประกอบ $S \circ T$ ด้วยการคูณเมทริกซ์ (หรือคำนวณทีละขั้น)

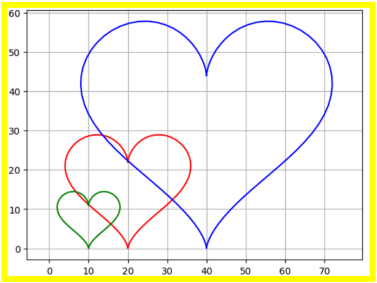
```python
import numpy as np

def transform(A, X):
    return np.dot(A, X)

A = np.array([[1, 1],
              [0, -1]])          # T
B = np.array([[2, -1],
              [-1, 1]])          # S

X = np.column_stack([[2, 3], [0, 1], [0, 5], [-1, 6]])

ax = transform(A, X)      # T(X)
bx = transform(B, ax)     # S(T(X))

# หรือทำครั้งเดียวด้วย BA
bx2 = transform(np.dot(B, A), X)

print("S∘T maps\n", X, "\nto\n", bx)
print("\nCheck with BA gives\n", bx2)
```

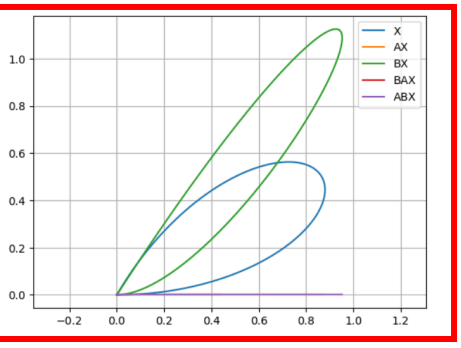
```python
# การหมุน
t = np.pi/6
A2 = np.array([[np.cos(t), -np.sin(t)], [np.sin(t), np.cos(t)]])
theta3 = np.linspace(0 , 2 * np.pi, 100)
X_v3 = 16 * (np.sin(theta3) ** 3) + 20
Y_v3 = 13 * np.cos(theta3) - 5 * np.cos(2 * theta3) - 2 * np.cos(3 * theta3) - np.cos(4 * theta3) + 17
X3 = np.row_stack((X_v3, Y_v3))
ax3 = tranform(A2, X3)
plt.axis("equal")
plt.grid(True)
plt.plot(X3[0,:], X3[1,:])
plt.plot(ax3[0,:], ax3[1,:])
plt.show()
```

```python
sp.pprint(simplify(U * U.T))  #U*U.T
sp.pprint(simplify(V.T * V))  #V.T * V
```

```python
import numpy as np
import matplotlib.pyplot as plt
def transform(M, X):
    return np.dot(M, X)
theta = np.linspace(0, np.pi/3, 100)
xvals = np.cos(theta) * np.sin(3*theta)
yvals = np.sin(theta) * np.sin(3*theta)
X = np.row_stack((xvals, yvals))

A = np.array([[1, 0],
              [0, 0]], dtype=float)

B = np.array([[0.5, 1],
              [0,   2]], dtype=float)

BA = B @ A
AB = A @ B

plt.axis("equal")
plt.grid(True)
for name, Y in datasets.items():
    plt.plot(Y[0, :], Y[1, :], label=name)

plt.legend()
plt.show()

print("A=\n", A)
print("B=\n", B)
print("BA=\n", BA)
print("AB=\n", AB)
```

```python
# การเปลี่ยนขนาด
def tranform(A, X):
    return np.dot(A,X)
A = np.array([[0.5,0],[0,0.5]])
B = np.array([[2,0],[0,2]])
theta1 = np.linspace(0,2*np.pi, 100)
X_v = 16 * (np.sin(theta1) ** 3) + 20
Y_v = 13 * np.cos(theta) - 5 * np.cos(2 * theta1) - 2 * np.cos(3 * theta1) - np.cos(4 * theta1) + 17
X = np.row_stack((X_v, Y_v))
ax = tranform(A, X)
bx = tranform(B, X)
plt.axis("equal")
plt.grid(True)
plt.plot(X[0,:], X[1,:], "r")
plt.plot(ax[0,:], ax[1,:], "g")
plt.plot(bx[0,:], bx[1,:], "b")
plt.show()
```






```python
# การสะท้อนด้านตรงข้าม
A1 = np.array([[1,0],[0,-1]])
theta2 = np.linspace(0,2 * np.pi, 100)
X_v2 = 16 * (np.sin(theta2) ** 3) + 20
Y_v2 = 13 * np.cos(theta2) - 5 * np.cos(2 * theta2) - 2 * np.cos(3 * theta2) - np.cos(4 * theta2) + 17
X2 = np.row_stack((X_v2, Y_v2))
ax2 = tranform(A1, X2)
plt.axis("equal")
plt.grid(True)
plt.plot(X2[0,:], X2[1,:])
plt.plot(ax2[0,:], ax2[1,:])
plt.show()
```

```python
dets = A6.det()
if product == dets:
    print("equals")
else:
    print("not equals")
```

```python
# การสะท้อนข้ามแกน Y นำผลที่ได้ไปสะท้อนต่อข้ามแกน X
A1 = np.array([[-1,0],[0,-1]])
theta2 = np.linspace(0,2 * np.pi, 100)
X_v2 = 16 * (np.sin(theta2) ** 3) + 20
Y_v2 = 13 * np.cos(theta2) - 5 * np.cos(2 * theta2) - 2 * np.cos(3 * theta2) - np.cos(4 * theta2) + 17
X2 = np.row_stack((X_v2, Y_v2))
ax2 = tranform(A1, X2)
plt.axis("equal")
plt.grid(True)
plt.plot(X2[0,:], X2[1,:])
plt.plot(ax2[0,:], ax2[1,:])
plt.show()
```

```python
# การหมุนแบบทวนเข็ม
theta3 = np.linspace(0 , 2 * np.pi, 100)
X_v3 = 16 * (np.sin(theta3) ** 3) + 20
Y_v3 = 13 * np.cos(theta3) - 5 * np.cos(2 * theta3) - 2 * np.cos(3 * theta3) - np.cos(4 * theta3) + 17
X3 = np.row_stack((X_v3, Y_v3))
plt.axis("equal")
plt.grid(True)
for i in range(8):
    t = (np.pi/6) * i
    A2 = np.array([[np.cos(t), -np.sin(t)], [np.sin(t), np.cos(t)]])
    ax3 = tranform(A2, X3)
    plt.plot(ax3[0,:], ax3[1,:])
plt.show()
```

```python
import numpy as np
import sympy as sp
from numpy import linalg as LA
from sympy import *

# ตรวจสอบ vecter ลักษณะเฉพาะที่เป็นอิสระเชิงเส้นต่อกัน
A1 = sp.Matrix([[2,1],[1,2]])
A1.eigenvects()
```

```python
import numpy as np
import sympy as sp
from numpy import linalg as LA
from sympy import *
```

```python
#หมุนตามเข็มนาฬิกา จำนวน 6 ครั้ง
def rotation_matrix(theta):
    return np.array([
        [np.cos(theta), np.sin(theta)],
        [-np.sin(theta), np.cos(theta)]
    ])

def tranform(A, X):
    return np.dot(A, X)


theta4 = np.linspace(0, 2 * np.pi, 100)
X_v4 = 16 * (np.sin(theta4) ** 3) + 20
Y_v4 = 13 * np.cos(theta4) - 5 * np.cos(2 * theta4) - 2 * np.cos(3 * theta4) - np.cos(4 * theta4) + 17
X4 = np.row_stack((X_v4, Y_v4))
plt.axis("equal")
plt.grid(True)

plt.plot(X4[0, :], X4[1,:], label="Original")
for i in range(1, 7):
    angle = i * (np.pi / 6)
    R = rotation_matrix(angle)
    rotated = tranform(R, X4)
    plt.plot(rotated[0, :], rotated[1,:], label=f"Rotate {i} pi / 6")

plt.legend()
plt.show()
```

```python
# ตรวจสอบสมการ Ax = Lamda X
for eigval, mult, vects in A1.eigenvects():
    v = vects[0]
    print(eigval, A1 * v, eigval * v, sp.simplify(A1*v - eigval*v))
```

การคำนวณ	numpy	sympy
การเรียก package	import numpy as np from numpy import linalg as LA	from sympy import *
การคำนวณค่าลักษณะเฉพาะของ A	eival, eivec = LA.eig(A)	A.eigenvals()
การคำนวณเวกเตอร์ลักษณะเฉพาะของ A	eival, eivec = LA.eig(A)	A.eigenvects แสดงผล (ค่าลักษณะเฉพาะ, จำนวนซ้ำ, [เวกเตอร์ลักษณะเฉพาะ])

```python
import sympy as sp
def check_diagonalizable(A):
A = sp.Matrix(A)
n = A.shape[0]
evects = A.eigenvects()
total_eigenvectors = 0
print("A=")
sp.pprint(A)
sp.pprint(evects)
for lam, alg_mult, basis in evects:
total_eigenvectors += len(basis)

print("Total number of independent eigenvectors =", total_eigenvectors)
print("Matrix size n =", n)
print()

if total_eigenvectors == n:
print("A is diagonalizable.")
else:
print("A is NOT diagonalizable.")


# example of use

A = sp.Matrix([[3,1,0],[0,3,0], [2,0,-1]])
check_diagonalizable(A)
```

```python
# แปลง เมทริกทแยงมุม หา P and D  A = PDP^-1
A2 = sp.Matrix([
    [4,0,1],
    [2,3,2],
    [1,0,4]
])
P, D = A2.diagonalize()
sp.pprint(P)
sp.pprint(D)
(P ** -1)*A2*P
```

```python
A6 = sp.Matrix([
    [1,2,3,4],
    [5,6,7,8],
    [9,10,11,12],
    [13,14,15,16]
])
# แปลงทแยงมุมได้หรือไม่
if A6.is_diagonalizable():
    P4, D4 = A6.diagonalize()
    sp.pprint(D4)
    II = (P4 ** -1)*A6*P4
    print(II.equals(D4))
else:
     print("Error diagonalize")
```

```python
# แต่ละ i = 1, 2, 3, 4, 5 คำนวนเวกเตอร์ลักษณะเฉพาะ vi ของ A ที่สมนัยกับ λi
eigen_info = A6.eigenvects()
count = 1
for val, mult, vectors in eigen_info:
    for v in vectors:
        print(sp.N(val, 3) ,list(v))
        count+=1
```

```python
# แต่ละ i = 1, 2, 3, 4, 5 คำนวน Avi และ λivi พร้อมวิเคราะห์ว่าสัมพันธ์กันอย่างไร
data = A6.eigenvects()
idx = 1
for lam, mult, vects in data:
    for v in vects:
        print(lam)
        sp.pprint(v)
        idx += 1
```

```python
# A * vi = si * ui
for i in range(S.rows):
    vi = V.col(i)
    ui = U.col(i)
    si = S[i, i]

    LHS = sp.simplify(A1 * vi)
    RHS = sp.simplify(A1 * ui)
    sp.pprint(LHS)
    sp.pprint(RHS)
    if((A1 * vi) == (si * ui)):
        print(True)
    else:
        print(False)
```

```python
# A.T * ui = si * vi
for i in range(S.rows):
    vi = V.col(i)
    ui = U.col(i)
    si = S[i, i]

    LHS = sp.simplify(A1 * vi)
    RHS = sp.simplify(A1 * ui)
    sp.pprint(LHS)
    sp.pprint(RHS)
    if((A1.T * ui) == (si * vi)):
        print(True)
    else:
        print(False)
```

```python
A1 = sp.Matrix([
    [1,2,3],
    [4,5,6],
    [7,8,9]
])
U, S,V = A1.singular_value_decomposition()
sp.pprint(simplify(U))  #left
sp.pprint(simplify(S))  #singular Value เป็นเมทริกทแยงมุม
sp.pprint(simplify(V))  #right
sp.pprint(simplify(U * S * V.T))
```

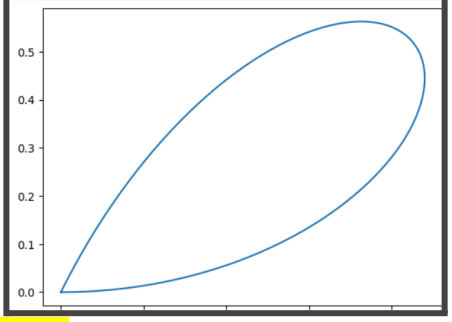
```python
# สะท้อนรูปหัวใจไปด้านตรงข้าม
A4 = np.array([[1,0],[0,-1]])
theta = np.linspace(0,2 * np.pi, 100)
xval1 = 16 * (np.sin(theta)**3) + 20
yval1 = 13 * np.cos(theta) - 5 * np.cos(2*theta) - 2 * np.cos(3*theta) - np.cos(4*theta) + 17
X2 = np.row_stack((xval1, yval1))
ax1 = tranform(A4,X2)
plt.axis("equal")
plt.grid(True)
plt.plot(X2[0,:],X2[1,:],"r")
plt.plot(ax1[0,:],ax1[1,:],"g")
plt.show()
```

```python
# สะท้อนข้ามแกน' แล้วน่าผลที่ได้ไปหมุนทวนเข็มนาฬิกา pi / 3
def tranform(A, X):
    return np.dot(A,X)

theta1 = np.linspace(0,2*np.pi, 100)
X_v = 16 * (np.sin(theta1) ** 3) + 20
Y_v = 13 * np.cos(theta) - 5 * np.cos(2 * theta1) - 2 * np.cos(3 * theta1) - np.cos(4 * theta1) + 17
X = np.row_stack((X_v, Y_v))

M2 = np.array([[-1, 0], [0, 1]])

R = np.array([
    [np.cos(np.pi / 3), -np.sin(np.pi / 3)],
    [np.sin(np.pi / 3), np.cos(np.pi / 3)]
])

# สะท้อน y ก่อนแล้วค่อยหมุน
step1 = tranform(M2, X)
result_a = tranform(R, step1)
plt.axis("equal")
plt.grid(True)
plt.plot(X[0,:], X[1,:], label = "Original")
plt.plot(step1[0,:], step1[1,:], label = "after")
plt.plot(result_a[0,:], result_a[1, :], label = "Rotated")
plt.legend()
plt.show()
```

```python
def tranform(A, X):
    return np.dot(A,X)
```

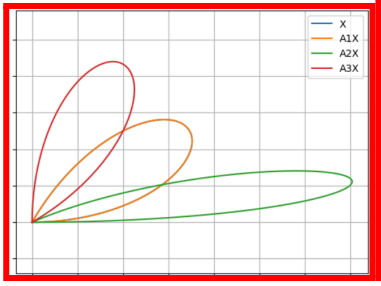
```python
theta1 = np.linspace(0,np.pi/3, 100)
xval2 = np.cos(theta1) * np.sin(3 * theta1)
yval2 = np.sin(theta1) * np.sin(3 * theta1)
X3 = np.row_stack((xval2, yval2))
plt.plot(xval2, yval2)
plt.show()
```

```
C:\Users\piyaw\AppData\Local\Temp\ipykernel_7876\2478191622.py:4: Deprecat
  X3 = np.row_stack((xval2, yval2))
```




```python
# เช็คว่าทแยงมุมได้หรือไม่ ถ้าได้ให้หา D และ P
A4 = sp.Matrix([
    [1,1,0],
    [0,1,1],
    [0,0,4]
])
if A4.is_diagonalizable():
    P2, D2 = A4.diagonalize()
    print(D2)
else:
    print("Error diagonalize")
```

```python
import numpy as np
import matplotlib.pyplot as plt

def transform(M, X):
    return np.dot(M, X)

theta = np.linspace(0, np.pi/3, 100)
xvals = np.cos(theta) * np.sin(3*theta)
yvals = np.sin(theta) * np.sin(3*theta)
X = np.vstack((xvals, yvals))

A = np.array([[0, -1],
              [1,  0]], dtype=float)

plt.axis("equal")
plt.grid(True)
plt.plot(X[0, :], X[1, :], label="X")

for p in [1, 2, 3]:
    Ap = np.linalg.matrix_power(A, p)
    Y = transform(Ap, X)
    plt.plot(Y[0, :], Y[1, :], label=f"A^{p}X")

plt.legend()
plt.show()
```




```python
import numpy as np
import matplotlib.pyplot as plt

def transform(M, X):
    return np.dot(M, X)

theta = np.linspace(0, np.pi/3, 100)
xvals = np.cos(theta) * np.sin(3*theta)
yvals = np.sin(theta) * np.sin(3*theta)
X = np.vstack((xvals, yvals))

As = [
    np.array([[1, 0],
              [0, 1]], dtype=float),      # I
    np.array([[2, 0],
              [0, 0.5]], dtype=float),    # scale
    np.array([[0, 1],
              [1, 0]], dtype=float),      # swap
]

plt.axis("equal")
plt.grid(True)
plt.plot(X[0, :], X[1, :], label="X")

for i, A in enumerate(As, 1):
    Y = transform(A, X)
    plt.plot(Y[0, :], Y[1, :], label=f"A{i}X")

plt.legend()
plt.show()
```